How the commands work

This project answers questions about companies' annual reports (10-K filings) using retrieval-augmented generation (RAG): instead of asking Claude to answer from memory, it first finds the relevant passages in the actual filings and gives those to Claude, which then answers from them and cites where each fact came from.

To make that possible, the filings have to be prepared in advance. That preparation is split into three commands, and a fourth command answers questions:

```
download  ->  data/raw/              the original 10-K files from the SEC
parse     ->  data/processed/        each filing split into its sections (Items)
index     ->  data/vectorstore/      searchable chunks: by meaning ...
              data/keyword_index.db  ... and by keyword
ask       ->  an answer with sources, using the two indexes
```

Each step saves its result to disk and the next step reads it. This means you can stop after any step, look at what it produced, and rerun only the steps affected by a change. It also keeps the cheap steps separate from the one that costs money: only `index` calls a paid API.

> On this machine, Windows Smart App Control blocks `sec-rag.exe`. Run every command as `python -m sec_rag.cli <command>` instead, for example `python -m sec_rag.cli index`.

---

1. `sec-rag download` - get the filings

```
sec-rag download AAPL MSFT NVDA --years 3
```

Purpose: fetch the original 10-K documents from the SEC so everything after this works from local files.

Every US public company files a 10-K each year with the SEC, and the SEC publishes them for free on its EDGAR system. This command uses EDGAR's public API to find and save them.

What it does, step by step:

1. **Finds the company.** EDGAR identifies companies by a number called the CIK, not by ticker. The command downloads the SEC's ticker-to-CIK list once and looks each ticker up (AAPL -> CIK 0000320193). An unknown ticker stops with an error.
2. **Lists the company's filings.** It asks EDGAR for the company's recent filings and keeps only the 10-Ks, newest first, up to `--years` of them (3 by default).
3. **Saves two files per filing** in `data/raw/<TICKER>/`, named `<fiscal year>_<accession number>`:
   - `.html` - the filing itself, exactly as the SEC publishes it.
   - `.json` - a small "sidecar" with the details the rest of the pipeline needs: company name, ticker, fiscal year, filing date, accession number (the SEC's unique ID for the filing) and the link to the original.

Things worth knowing:

- **Safe to rerun.** Filings already on disk are skipped, so running it again only fetches what is new (for example, a 10-K published since the last run).
- **Fiscal year** is taken from the period the report covers, not the date it was filed. NVIDIA's year ends in late January, so its "FY2025" report covers mostly calendar 2024.
- **SEC rules.** The SEC allows at most 10 requests per second and requires every request to identify who is making it, with a contact email. The command pauses between requests, and the identification comes from `SEC_RAG_EDGAR_USER_AGENT` in `.env`.

---

2. `sec-rag parse` - turn HTML into sections

```
sec-rag parse
```

Purpose: turn each raw HTML filing into clean text, organised by the sections of the report.

A raw 10-K is a large HTML file (often several megabytes) full of formatting, tables, hidden XBRL tags and navigation. It is also highly structured: every 10-K is divided into standard numbered sections called **Items**, defined by the SEC:

Item	Section
1	Business
1A	Risk Factors
7	Management's Discussion and Analysis (MD&A)
7A	Market Risk
8	Financial Statements
...	(about 20 in total)

Knowing which Item a passage came from is very useful later: it tells Claude (and you) whether a sentence is a risk the company warns about or a result it reports.

What it does, step by step:

1. **Extracts the readable text.** It removes HTML tags, scripts and styling and keeps the text a person would see, with paragraph breaks preserved.
2. **Finds where each Item starts.** It looks for headings like "Item 1A. Risk Factors". Every 10-K also lists these headings in its table of contents near the top; the parser recognises that list and skips it, so a section starts at the real heading, not at the table of contents entry.
3. **Cuts the text into sections**, from one Item heading to the next. Tiny fragments (under 200 characters, usually a stray heading or a cross-reference) are dropped. If no Item headings are found at all, the whole filing is kept as one section, so nothing is lost.
4. **Saves one JSON file per filing** in `data/processed/`, for example `AAPL_2024_0000320193-24-000123.json`, holding the filing details and its list of sections (Item number, title and text).

Things worth knowing:

- **Free and local.** Nothing is sent anywhere; it takes a few seconds per filing.
- **Check the output here.** The JSON files are readable. Before paying for embeddings, it is worth opening one and checking that the sections look right (for example, that Item 1A really contains the risk factors and is not empty).
- **Rerun after parser changes.** `parse` always re-parses every downloaded filing and overwrites the JSON files, so improvements to the parser apply to all of them.

---

3. `sec-rag index` - make the filings searchable

```
sec-rag index
```

Purpose: prepare the parsed sections so that, given a question, the right passages can be found in a fraction of a second.

A whole section is far too long to search or to give to Claude as-is (Item 1A alone can be tens of thousands of words), so the sections are first cut into small pieces called **chunks**. Each chunk is then stored in two indexes that search in different ways.

What it does, step by step:

1. **Reads the parsed filings** from `data/processed/`. If a downloaded filing has not been parsed yet, it is skipped and named in the output, so you know to run `sec-rag parse`. If nothing has been parsed at all, the command stops before spending anything.

2. **Chunks each section.** Text is packed paragraph by paragraph into chunks of about 1,500 characters (roughly 250 words). Two details make the chunks more useful:

    - **Overlap.** Each chunk repeats the last 200 characters of the one before it, so a sentence that falls on a boundary still appears whole in at least one chunk.
    - **A header on every chunk**, such as `Apple Inc. (AAPL) 10-K FY2024 - Item 1A: Risk Factors`. A chunk on its own might just say "our revenue grew 6%"; the header tells the search, and later Claude, whose revenue, which year and which section.

    Chunks never cross from one Item into another.

3. **Embeds each chunk.** An *embedding* is a list of numbers that represents what a piece of text means. Texts with similar meaning get similar numbers, even when they use different words ("supply chain disruption" and "shortages of components" end up close together). The chunks are sent, 128 at a time, to Voyage AI's `voyage-finance-2` model, which is trained on financial text and returns 1,024 numbers per chunk. This is the only step that costs money; if Voyage asks the client to slow down (rate limiting), the command waits and retries automatically.

4. **Stores every chunk in two indexes:**

    - **Vector store** (ChromaDB, in `data/vectorstore/`): the chunk text, its embedding and its details (ticker, year, Item). Used for **search by meaning**.

- **Keyword index** (SQLite full-text search, in `data/keyword_index.db`): the chunk's words. Used for **search by exact terms**. Meaning-based search can miss exact names and codes such as "H100", "CUDA" or "Vision Pro"; keyword search finds them reliably.

Things worth knowing:

- **Clean replacement.** When a filing is indexed again, its old chunks are deleted first. Without this, a change that produces fewer chunks would leave old ones behind.
- **Changing the embedding model rebuilds everything.** Embeddings from different models are not comparable (they do not even have the same length). The vector store remembers which model built it; if that differs from the configured model, `index` clears it and re-embeds every filing. Asking a question against a store built with another model gives a clear error telling you to run `index`.
- **Cost.** Indexing about 2,300 chunks (9 filings) is roughly a million tokens, which is within Voyage's free allowance.
- `--keyword-only` rebuilds just the keyword index from chunks already in the vector store. It calls no API and costs nothing.

---

4. `sec-rag ask` - answer a question

```
sec-rag ask "What supply chain risks does AAPL describe in FY2024?"
```

Purpose: find the passages that answer the question and have Claude write an answer from them, with citations.

What it does, step by step:

1. **Reads filters from the question.** Tickers written in capitals that exist in the index (`AAPL`, `MSFT`) and years (`2024`, `FY2024`) narrow the search to those filings. Writing the company name ("Apple") does not set a filter, so use the ticker when you want to be precise. You can also set filters directly with `--ticker AAPL --year 2024`.
2. **Searches both indexes.** The question is embedded with the same Voyage model (in its "query" mode) and compared with the stored chunks by meaning; at the same time the keyword index is searched for its words. The two ranked lists are merged with *Reciprocal Rank Fusion*: a chunk ranked high in both lists rises to the top.
3. **Handles comparisons fairly.** A question naming several companies or years (for example "Compare AAPL and MSFT R&D spending in 2024") is searched once per company/year, and the results are interleaved, so each one gets its share of chunks instead of the best-matching company taking them all.
4. **Reranks (optional).** If `SEC_RAG_RERANK=true`, Claude reads the question together with each of the top candidate chunks, scores how useful each one is, and only the best are kept. This is slower but more precise than the search scores alone.
5. **Writes the answer.** The question and the chosen chunks (8 by default, `--top-k` to change) are sent to Claude, which answers using only those passages and cites them. The answer is printed as it is written.
6. **Lists the sources**: company, fiscal year, Item and relevance score for each chunk used. The scores are merged rank scores (small numbers like 0.03), not percentages.

When to rerun what

You changed...	Run
Added companies or years	`download`, then `parse`, then `index`
The parser code	`parse`, then `index`
Chunk size, overlap, or the embedding model	`index`
Nothing - just asking questions	`ask`